Saga

Order booking: orchestration + event-bus choreography with compensations

Today's Hook

- Problem: Order booking: orchestration + event-bus choreography with...
- Naive pain cue: --- naive (payment fails → inventory leak) ---
- We will refactor to Saga with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Book an order: reserve inventory → charge payment → create shipment.
- Naive sequential calls leak reservations when payment fails.
- Orchestration saga registers compensations; reverse on abort.
- Choreography twin: EventBus InventoryReserved → PaymentCharged → ShipmentCreated.

Structure



Saga: Order booking: orchestration + event-bus choreography with...

Real-World Scenario

- Inputs: orderId, sku, qty, cents, address.
- Success: log contains committed; partners.shipped contains orderId.
- Failure: ship/charge fails → compensate refund then release (reverse).
- Choreography: StepFailed event triggers InventoryReleased on the bus.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.saga.problem;

/**
 * Naive: sequential calls with no compensation registry.
 * If shipping fails after payment, money is taken and inventory stays reserved.
 */
public final class NaiveOrderFlow {
    private final OrderBookingSaga.InMemoryPartners partners;

    public NaiveOrderFlow(OrderBookingSaga.InMemoryPartners partners) {
        this.partners = partners;
    }

    public boolean book(String orderId, String sku, int qty, int cents, String address) {
        var r1 = partners.reserve(orderId, sku, qty);
        if (!r1.ok()) return false;
        var r2 = partners.charge(orderId, cents);
        if (!r2.ok()) return false; // inventory still reserved — leak
        var r3 = partners.ship(orderId, address);
        return r3.ok(); // payment charged even if ship fails — leak
    }
}
```

Why Naive Hurts

- NaiveOrderFlow leaves inventory reserved after card decline.
- No compensation registry — money/stock drift on mid-flow failure.
- Cannot inject 'fail at ship' without rewriting the flow.
- No idempotency key — retries double-charge in production.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.saga.problem;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.function.Supplier;

/**
 * Orchestration saga for: reserve inventory → charge payment → create shipment.
 * Each step registers a compensation; on failure compensations run in reverse order.
 * Steps are idempotent via a simple seen-set keyed by orderId+step.
 */
public final class OrderBookingSaga {
    public record StepResult(boolean ok, String detail) {}

    public interface Inventory { StepResult reserve(String orderId, String sku, int qty); void release(String orderId); }
    public interface Payments { StepResult charge(String orderId, int cents); void refund(String orderId); }
    public interface Shipping { StepResult ship(String orderId, String address); void recall(String orderId); }

    private final Inventory inventory;
    private final Payments payments;
    private final Shipping shipping;
    private final List<String> log = new ArrayList<>();
    private final List<String> seen = new ArrayList<>();
```

Solution (2)

```
public OrderBookingSaga(Inventory inventory, Payments payments, Shipping shipping) {
    this.inventory = inventory;
    this.payments = payments;
    this.shipping = shipping;
}

public List<String> log() { return List.copyOf(log); }

public boolean book(String orderId, String sku, int qty, int cents, String address) {
    List<Runnable> compensations = new ArrayList<>();
    try {
        runOnce(orderId + ":reserve", () -> {
            var r = inventory.reserve(orderId, sku, qty);
            if (!r.ok()) throw new IllegalStateException("inventory:" + r.detail());
            log.add("ok:reserve");
            compensations.addFirst(() -> { inventory.release(orderId); log.add("compensate:release"); });
        });
        runOnce(orderId + ":charge", () -> {
            var r = payments.charge(orderId, cents);
            if (!r.ok()) throw new IllegalStateException("payment:" + r.detail());
            log.add("ok:charge");
            compensations.addFirst(() -> { payments.refund(orderId); log.add("compensate:refund"); });
        });
        runOnce(orderId + ":ship", () -> {
            var r = shipping.ship(orderId, address);
            if (!r.ok()) throw new IllegalStateException("shipping:" + r.detail());
            log.add("ok:ship");
            compensations.addFirst(()
```

Runnable main

```java
package com.buildwithwaqar.patterns.enterprise.saga.problem;

/** Problem: reserve → charge → ship with reverse compensations (orchestration + choreography). */
public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Saga problem: order booking with rollback ===");
        System.out.println("--- naive (payment fails → inventory leak) ---");
        var leak = new OrderBookingSaga.InMemoryPartners(() -> true, () -> false, () -> true);
        System.out.println("naiveBooked=" + new NaiveOrderFlow(leak).book("N1", "SKU", 1, 100, "A"));
        System.out.println("naiveReserved=" + leak.reserved());

        System.out.println("--- orchestration happy path ---");
        var okP = OrderBookingSaga.InMemoryPartners.alwaysOk();
        var ok = new OrderBookingSaga(okP, okP, okP);
        System.out.println("booked=" + ok.book("S1", "SKU", 1, 1000, "Ship St"));
        System.out.println("log=" + ok.log());

        System.out.println("--- orchestration fails at ship; compensates reverse ---");
        var failP = new OrderBookingSaga.InMemoryPartners(() -> true, () -> true, () -> false);
        var fail = new OrderBookingSaga(failP, failP, failP);
        System.out.println("booked=" + fail.book("S2", "SKU", 1, 1000, "Ship St"));
        System.out.println("log=" + fail.log());
```

Runnable main (2)

```
        System.out.println("--- choreography happy path (event bus) ---");
        var bus = new EventBus();
        var cOk = OrderBookingSaga.InMemoryPartners.alwaysOk();
        var choreo = new ChoreographyOrderSaga(bus, cOk);
        System.out.println("booked=" + choreo.book("C1", "SKU", 1, 500, "Bus Ave"));
        System.out.println("log=" + choreo.log());
        System.out.println("events=" + bus.history().stream().map(e -> e.getClass().getSimpleName()).toList());

        System.out.println("--- choreography fails at charge; compensates ---");
        var bus2 = new EventBus();
        var cFail = new OrderBookingSaga.InMemoryPartners(() -> true, () -> false, () -> true);
        var choreoFail = new ChoreographyOrderSaga(bus2, cFail);
        System.out.println("booked=" + choreoFail.book("C2", "SKU", 1, 500, "Bus Ave"));
        System.out.println("log=" + choreoFail.log());
        System.out.println("reserved=" + cFail.reserved() + " charged=" + cFail.charged());
    }
}
```

Walkthrough Output

```
=== Saga problem: order booking with rollback ===
--- naive (payment fails → inventory leak) ---
naiveBooked=false
naiveReserved=[N1]
--- orchestration happy path ---
booked=true
log=[ok:reserve, ok:charge, ok:ship, committed]
--- orchestration fails at ship; compensates reverse ---
booked=false
log=[ok:reserve, ok:charge, abort:shipping:carrier_down, compensate:refund, compensate:release, rolled_back]
--- choreography happy path (event bus) ---
booked=true
log=[ok:reserve, react:payment-on-reserved, ok:charge, react:ship-on-charged, ok:ship, committed]
events=[InventoryReserved, PaymentCharged, ShipmentCreated]
--- choreography fails at charge; compensates ---
booked=false
log=[ok:reserve, react:payment-on-reserved, abort:charge:card_declined, compensate:release, rolled_back]
reserved=[] charged=[]
```

Step-by-Step Trace

- Match stdout to stages — look for: booked=true
- Then the injected failure: --- naive (payment fails → inventory leak) ---
- Re-run: java -cp … 33-saga ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.enterprise.saga.problem;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.function.Supplier;

/**
 * Orchestration saga for: reserve inventory → charge payment → create shipment.
 * Each step registers a compensation; on failure compensations run in reverse order.
 * Steps are idempotent via a simple seen-set keyed by orderId+step.
 */
public final class OrderBookingSaga {
    public record StepResult(boolean ok, String detail) {}

    public interface Inventory { StepResult reserve(String orderId, String sku, int qty); void release(String orderId); }
    public interface Payments { StepResult charge(String orderId, int cents); void refund(String orderId); }
    public interface Shipping { StepResult ship(String orderId, String address); void recall(String orderId); }

    private final Inventory inventory;
    private final Payments payments;
    private final Shipping shipping;
    private final List<String> log = new ArrayList<>();
    private final List<String> seen = new ArrayList<>();
```

Modern Java 25 / Zazr (2)

```
    public OrderBookingSaga(Inventory inventory, Payments payments, Shipping shipping) {
        this.inventory
```

Trade-offs

- Use Saga when «Order booking: orchestration + event-bus choreography with compensations» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Saga in product language.

Related Patterns

- Name the pattern usually confused with Saga and separate the forces.
- Sketch the roles for «Order booking: orchestration + event-bus choreography with compensations» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Order booking: orchestration + event-bus choreography with compensations
- Naive pain → Saga boundary → Demo stdout.
- Remember the failure cue: --- naive (payment fails → inventory leak) ---
- Break a ProblemTest assertion, watch it fail, restore it.